

FLOODRIX

Predict. Protect. Deliver.

API Specification & Backend–Flutter Integration Document

AI-Powered Flood Prediction and Resilient Logistics Routing System

Mobile Application: Flutter
Backend: Python + FastAPI
API Version: v1
Document Version: 1.0

1. Purpose

This document defines the API contract for the FLOODRIX mobile application and Python backend. The Flutter application communicates with the FastAPI backend; the backend is responsible for routing, weather integration, flood prediction, risk calculation, ETA estimation, road-closure handling, alerts, trips, and simulation.

The API is designed around FLOODRIX's core workflow: obtain a route, evaluate flood probability along the route using the trained flood model, combine that risk with logistics constraints, and return a practical route or decision to the mobile client.

FastAPI supports path operations using HTTP methods such as GET, POST, PUT, and DELETE, and APIRouter can be used to organize endpoints into separate modules. ■cite■turn0search1■turn0search3■

Important implementation principle: Flutter should not directly call the weather provider or execute the ML model. It should communicate with the FLOODRIX backend.

2. High-Level Architecture

```
Flutter Mobile App
|
| HTTPS / JSON
v
FLOODRIX FastAPI Backend
|
+-- Routing Service
+-- Weather Service
+-- Flood Prediction Service
+-- Risk Engine
+-- ETA Service
+-- Closure Service
+-- Alert Service
+-- Simulation Engine
|
+-- Database
+-- Trained ML Model
+-- External Weather / Routing Providers
```

3. Base URL and Versioning

```
Production:
https://api/v1

Development:
http://localhost:8000/api/v1
```

All mobile-facing endpoints should be versioned under **/api/v1**. This allows a future v2 contract without breaking the existing Flutter application.

4. Authentication

Authentication is not required for the first hackathon MVP unless a multi-user account system is introduced. If authentication is added, protected endpoints should accept an access token through the Authorization header.

```
Authorization: Bearer
```

5. MVP Endpoint Summary

Method	Endpoint	Purpose
GET	/health	Backend/model health check
POST	/routes/plan	Plan shipment and select safest practical route

Method	Endpoint	Purpose
POST	/prediction/flood	Predict flood probability for one location
POST	/prediction/route	Predict flood risk across route segments
GET	/weather/current	Get current weather/rainfall inputs
GET	/weather/forecast	Get forecast rainfall/weather
GET	/map/risk	Get risk data for map viewport
GET	/closures	Get active road closures
POST	/trips	Create a planned trip
POST	/trips/{trip_id}/location	Send live trip location
GET	/alerts/active	Get active alerts
POST	/simulation/run	Run what-if flood/closure simulation

6. Endpoint Specifications

6.1 Health Check

```
GET /api/v1/health
```

Purpose: Verify that the backend is reachable and that the flood model is loaded.

Response:

```
{
  "status": "ok",
  "service": "FLOODRIX",
  "version": "1.0.0",
  "model_loaded": true
}
```

Implementation notes: Flutter can use this endpoint during development and diagnostics.

6.2 Plan Route / Shipment

```
POST /api/v1/routes/plan
```

Purpose: Create a route plan from an origin to a destination and return the recommended route plus alternatives.

Request:

```
{
  "origin": {"latitude": 10.7867, "longitude": 76.6548},
  "destination": {"latitude": 9.9312, "longitude": 76.2673},
  "cargo_type": "perishable",
  "departure_time": "2026-10-07T11:00:00+05:30"
}
```

Response:

```
{
  "recommended_route": {
    "route_id": "R1",
    "distance_km": 178.4,
    "base_eta_minutes": 265,
    "risk_score": 0.21,
    "risk_level": "LOW",
    "eta_best_minutes": 265,
    "eta_worst_minutes": 310
  },
  "alternatives": [
    {
      "route_id": "R2",
      "distance_km": 184.2,
      "base_eta_minutes": 280,
      "risk_score": 0.12,
      "risk_level": "LOW",
      "eta_best_minutes": 280,
      "eta_worst_minutes": 300
    }
  ],
  "decision": "R1"
}
```

Implementation notes: The backend should call the routing provider, obtain candidate routes, evaluate flood risk, check closures, calculate ETA, and apply the logistics decision engine.

6.3 Single-Location Flood Prediction

```
POST /api/v1/prediction/flood
```

Purpose: Run the trained flood model for one geographic point.

Request:

```
{
  "latitude": 10.0159,
  "longitude": 76.3419,
  "rainfall_1h": 18.4,
  "rainfall_6h": 62.7,
  "rainfall_24h": 141.2,
}
```

```
"elevation": 12.5,
"historical_flood_frequency": 0.72,
"flood_zone": 1
}
```

Response:

```
{
  "latitude": 10.0159,
  "longitude": 76.3419,
  "flood_probability": 0.83,
  "risk_level": "HIGH",
  "model_version": "floodrix-v1",
  "prediction_time": "2026-10-07T10:30:00+05:30"
}
```

Implementation notes: The ML target is Flood_occurred. The prediction request does not include the future Flood_occurred value.

6.4 Route Flood Prediction

POST /api/v1/prediction/route

Purpose: Predict flood probability for multiple points/segments along a route.

Request:

```
{
  "route_id": "route_001",
  "segments": [
    {"latitude": 10.0159, "longitude": 76.3419},
    {"latitude": 10.0321, "longitude": 76.2982},
    {"latitude": 10.0552, "longitude": 76.2751}
  ]
}
```

Response:

```
{
  "route_id": "route_001",
  "overall_probability": 0.68,
  "risk_level": "HIGH",
  "segments": [
    {
      "latitude": 10.0159,
      "longitude": 76.3419,
      "flood_probability": 0.31,
      "risk": "LOW"
    },
    {
      "latitude": 10.0321,
      "longitude": 76.2982,
      "flood_probability": 0.67,
      "risk": "HIGH"
    }
  ]
}
```

Implementation notes: Flutter can use segment-level risk to render green/yellow/red route polylines.

6.5 Current Weather

GET /api/v1/weather/current?latitude=10.01&longitude=76.34

Purpose: Return current weather/rainfall values needed by the prediction layer.

Response:

```
{
  "latitude": 10.01,
  "longitude": 76.34,
  "rainfall_1h": 12.4,
  "rainfall_6h": 43.8,
  "rainfall_24h": 112.2,
  "temperature": 27.4,
  "weather_code": 65
}
```

Implementation notes: The backend should abstract the external weather provider so Flutter does not depend on the provider's API contract.

6.6 Weather Forecast

```
GET /api/v1/weather/forecast?latitude=10.01&longitude;=76.34
```

Purpose: Return future weather/rainfall values used for route-time prediction.

Response:

```
{
  "forecast": [
    {"time": "11:00", "rainfall": 4.2},
    {"time": "12:00", "rainfall": 8.5},
    {"time": "13:00", "rainfall": 15.3}
  ]
}
```

Implementation notes: Forecast values should be transformed into the same model input schema required by the prediction service.

6.7 Map Risk

```
GET /api/v1/map/risk?min_lat=9.8&max;_lat=10.2&min;_lon=76.1&max;_lon=76.5
```

Purpose: Return flood-risk information for the visible map area.

Response:

```
{
  "segments": [
    {
      "id": "SEG001",
      "geometry": [],
      "flood_probability": 0.12,
      "risk_level": "LOW"
    },
    {
      "id": "SEG002",
      "geometry": [],
      "flood_probability": 0.78,
      "risk_level": "HIGH"
    }
  ]
}
```

Implementation notes: Use a bounding box so the mobile app only requests data for the current map viewport.

6.8 Active Road Closures

```
GET /api/v1/closures
```

Purpose: Return currently known road closures/disruptions.

Response:

```
{
  "closures": [
    {
      "id": "CL001",
      "latitude": 10.023,
      "longitude": 76.321,
      "road_name": "NH 85",
      "severity": "BLOCKED",
      "source": "operator_report",
      "verified": false,
      "reported_at": "2026-10-07T09:20:00+05:30"
    }
  ]
}
```

Implementation notes: Unverified operator reports must remain explicitly labelled as unverified.

6.9 Create Trip

```
POST /api/v1/trips
```

Purpose: Create a trip after the operator selects a route.

Request:

```
{
  "origin": {"latitude": 10.7867, "longitude": 76.6548},
  "destination": {"latitude": 9.9312, "longitude": 76.2673},
  "route_id": "R1",
  "cargo_type": "perishable"
}
```

Response:

```
{
  "trip_id": "TRP_001",
  "status": "PLANNED"
}
```

6.10 Live Trip Location

POST /api/v1/trips/{trip_id}/location

Purpose: Send periodic GPS updates from Flutter during an active trip.

Request:

```
{
  "latitude": 10.4512,
  "longitude": 76.3921,
  "speed_kmh": 42.5,
  "timestamp": "2026-10-07T12:32:00+05:30"
}
```

Response:

```
{
  "trip_id": "TRP_001",
  "received": true,
  "current_risk_level": "MEDIUM",
  "reroute_required": false
}
```

Implementation notes: The backend can use the location to identify the current route segment and reevaluate current conditions.

6.11 Active Alerts

GET /api/v1/alerts/active

Purpose: Return active flood, closure, route-risk, or trip alerts.

Response:

```
{
  "alerts": [
    {
      "id": "ALT001",
      "type": "FLOOD_RISK",
      "severity": "HIGH",
      "title": "High flood probability ahead",
      "message": "High flood probability detected 8 km ahead.",
      "latitude": 10.12,
      "longitude": 76.34
    }
  ]
}
```

6.12 Simulation

POST /api/v1/simulation/run

Purpose: Run a what-if scenario using the same prediction and routing pipeline.

Request:

```
{
  "route_id": "R1",
  "rainfall_multiplier": 2.5,
  "inject_closure": true,
  "closure_segment": "SEG034"
}
```

Response:

```
{
  "simulation_id": "SIM001",
  "original_risk": 0.21,
  "simulated_risk": 0.79,
  "original_decision": "GO",
  "simulated_decision": "REROUTE"
}
```

Implementation notes: Simulation must modify real pipeline inputs rather than return hardcoded demonstration outputs.

7. Flood Prediction Contract

The core model uses the following fixed feature schema.

Feature	Role	Type
Latitude	Geographic input	Float
Longitude	Geographic input	Float
Rainfall_1h	Recent rainfall	Float
Rainfall_6h	Short-term accumulated rainfall	Float
Rainfall_24h	Daily accumulated rainfall	Float
Elevation	Terrain input	Float
Historical_flood_frequency	Historical flood occurrence signal	Float
Flood_zone	Historical/contextual flood-zone input	Integer/Encoded
Flood_occurred	Training target	0/1

At training time, historical input features are paired with the actual historical Flood_occurred target. At prediction time, the target is unknown and must never be sent by Flutter as an input.

8. Route Decision Flow

1. Flutter sends origin + destination + shipment information
2. Backend requests candidate routes
3. Backend divides routes into segments / prediction points
4. Weather service obtains current and forecast rainfall
5. Geographic service supplies elevation / historical frequency / flood-zone data
6. Flood model predicts probability for each segment
7. Closure service checks known disruptions
8. Risk engine calculates route risk
9. ETA service calculates best/worst ETA
10. Logistics decision engine selects GO / REROUTE / WAIT
11. Backend returns route options to Flutter

9. Logistics Layer

The flood ML model predicts flood probability. Logistics-specific parameters are applied after prediction. Cargo type, delivery deadline, ETA, and road closures therefore belong to the route decision layer rather than the flood-model feature vector.

Parameter	Used By	Purpose
Cargo type	Decision engine	Adjust risk/route penalty
Delivery deadline	Decision engine	Determine time sensitivity
ETA	ETA/decision layer	Estimate delivery impact
Road closure	Closure/decision layer	Prevent or penalize route
Flood probability	ML + decision layer	Measure predicted flood risk

10. Suggested Response Standards

Status	Meaning
200	Successful request

Status	Meaning
201	Resource created
400	Invalid request
401	Authentication required/invalid
404	Resource not found
422	Validation error
429	Rate limit exceeded
500	Internal backend error
503	External service/model unavailable

FastAPI can declare response status codes and response models as part of path-operation configuration, which also feeds its generated OpenAPI documentation. [cite](#)[turn0search2](#)

11. Backend Module Structure

```
floodrix-backend/
├── app/
│   ├── main.py
│   └── api/v1/
│       ├── routes.py
│       ├── prediction.py
│       ├── weather.py
│       ├── trips.py
│       ├── closures.py
│       ├── alerts.py
│       ├── simulation.py
│       ├── dashboard.py
│       └── services/
│           ├── routing_service.py
│           ├── weather_service.py
│           ├── flood_prediction_service.py
│           ├── risk_service.py
│           ├── eta_service.py
│           ├── closure_service.py
│           └── alert_service.py
│       ├── ml/
│       │   ├── model.pkl
│       │   ├── predictor.py
│       │   └── preprocessing.py
│       ├── models/
│       ├── schemas/
│       ├── core/
│       └── requirements.txt
```

Using separate APIRouter modules is consistent with FastAPI's recommended structure for larger applications and allows routes to be grouped by concern. [cite](#)[turn0search3](#)

12. Flutter Integration Rules

1. Flutter communicates only with the FLOODRIX backend for application data.
2. Use a single API client layer in Flutter.
3. Keep request/response DTOs aligned with the Pydantic schemas on the backend.
4. Store the backend base URL in configuration rather than hardcoding it throughout the app.
5. Handle timeout, 4xx, 5xx, and offline states explicitly.
6. Do not expose provider API keys in the Flutter application.
7. Do not place the ML model in Flutter for the MVP.

13. Recommended Build Order

Order	Backend Work	Flutter Work
1	Health + project skeleton	API client + environment config
2	Weather service	Weather/connection state
3	Flood prediction endpoint	Prediction response model
4	Routing + route planning	Plan Shipment screen
5	Route risk endpoint	Map + risk polylines
6	Closures	Closure markers
7	Trips + location	Live Trip screen
8	Alerts	Alert UI
9	Simulation	What-If Simulator
10	Integration testing	Final UI polish

14. OpenAPI / API Documentation

FastAPI automatically exposes OpenAPI-based API documentation for the registered path operations. This should be used as the source of truth while implementing the Flutter client. ■cite■turn0search0■turn0search2■

```
Development:
http://localhost:8000/docs

OpenAPI schema:
http://localhost:8000/openapi.json
```

15. Final API Contract

The MVP backend contract is intentionally focused. The critical path is: **Plan Route** → **Obtain Weather** → **Predict Flood Risk** → **Evaluate Closures** → **Calculate ETA** → **Select Route** → **Monitor Trip** → **Alert/Reroute**.

This contract keeps the ML prediction layer separate from the logistics decision layer while allowing the mobile application to consume one consistent backend interface.

16. References

FastAPI official documentation: path operations and HTTP methods. ■cite■turn0search1■

FastAPI official documentation: APIRouter and larger multi-file applications. ■cite■turn0search0■turn0search3■

FastAPI official documentation: path-operation configuration, response models, and status codes. ■cite■turn0search2■